

PENGURUTAN DATA MAHASISWA DENGAN METODE *MERGE SORT* BERBASIS *WEB STATIC*

Struktur Data

Disusun oleh:

20272313003 Michael Sihombing Kusuma

20272313010 Bintang Wijayanto Putro

20272313011 M. Kevin Hartono Putra

PROGRAMS OF STUDY COMPUTER SCIENCE

BINA NUSANTARA UNIVERSITY

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Maha Kuasa, karena atas segala limpahan rahmat dan karunia-Nya, penulis dapat menyelesaikan makalah ini dengan lancar. Segala puji bagi Tuhan, yang telah memberikan petunjuk dan kesempatan bagi kami untuk menyelesaikan makalah “Pengurutan Data Mahasiswa dengan Metode *Merge Sort* berbasis *Web Static*”.

Makalah ini membahas berbagai aspek yang terkait dengan *Merge Sort*, mulai dari definisi, langkah-langkah algoritma, kekurangan atau kelebihan, dan analisis kompleksitas waktu, hingga implementasi dalam berbagai bahasa pemrograman. Kami berharap makalah ini dapat menjadi referensi yang bermanfaat bagi para pembaca yang ingin memahami lebih dalam tentang *Merge Sort*.

Akhir kata, semoga makalah ini dapat memberikan manfaat dan kontribusi positif bagi perkembangan ilmu pengetahuan, khususnya di bidang algoritma dan struktur data.

Bandung, 10 Juni 2024

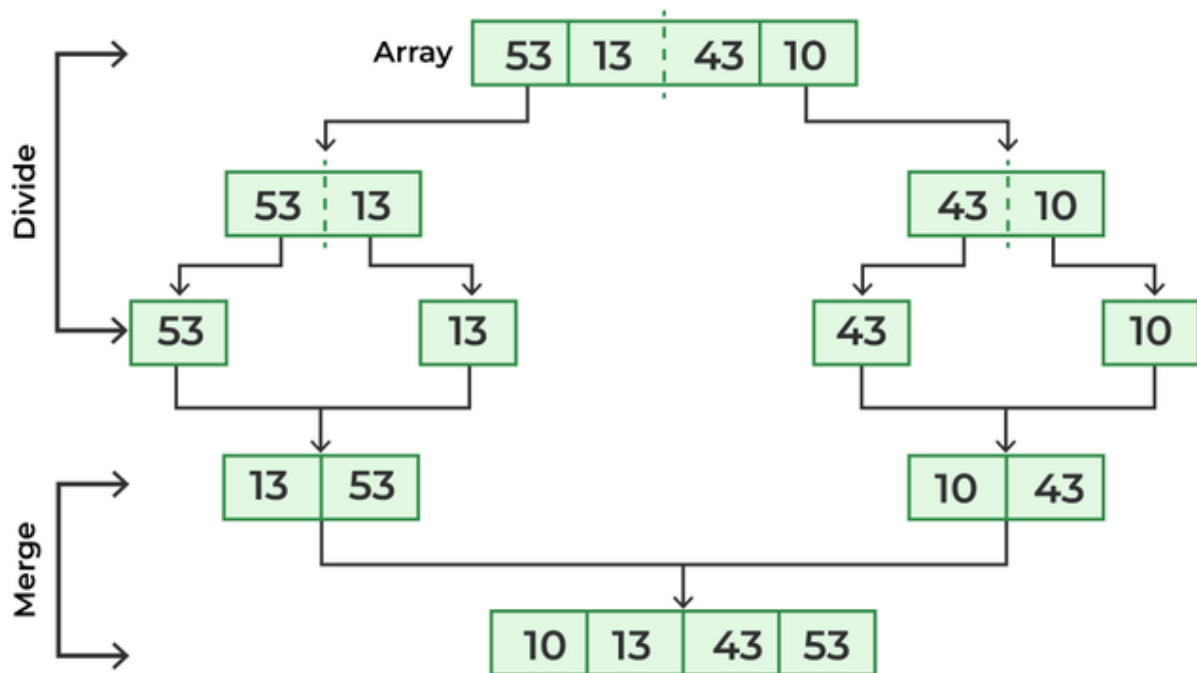
Penulis

DEFINISI

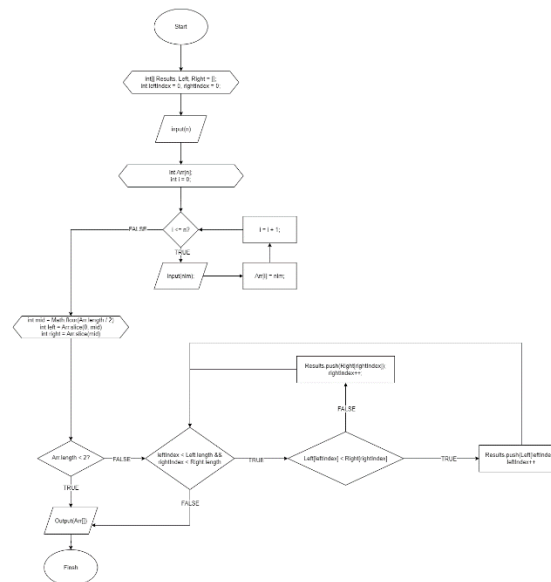
Algoritma Merge Sort adalah salah satu metode pengurutan data yang berbasis perbandingan dan memanfaatkan teknik “*Divide and Conquer*” atau “Bagi dan taklukkan”. Metode ini efisien untuk mengurutkan kumpulan data dengan ukuran besar. Berikut adalah penjelasan singkat mengenai algoritma:

1. Pemisahan (*Divide*): Langkah pertama adalah membagi daftar data yang akan diurutkan menjadi dua bagian. Ini dilakukan dengan menentukan elemen tengah dari daftar data sebagai titik pembagiannya.
2. Penaklukan (*Conquer*): Setelah pemisahan, masing-masing bagian data diurutkan secara terpisah menggunakan rekursi.
3. Penggabungan (*Merge*): Bagian-bagian yang telah diurutkan digabungkan kembali dalam urutan yang benar. Algoritma ini menggabungkan dua bagian data dengan cara membandingkan elemen-elemen mereka dan menyusunnya kembali.

ILUSTRASI



FLOWCHART



KELEBIHAN

1. Efisiensi pada Data Besar: Cocok untuk data dengan ukuran besar.
2. Penggunaan Memori: Memiliki penggunaan memori yang wajar.
3. Kasus Terburuk yang Konsisten: Kinerja tetap baik dalam kasus terburuk.
4. Pengurutan Linked List: Cocok untuk mengurutkan tautan berantai (*linked list*).

KEKURANGAN

1. Memerlukan alokasi memori tambahan untuk menyimpan bagian-bagian data.
2. Proses penggabungan memerlukan waktu dan sumber daya.
3. Tidak efisien untuk data yang sering berubah.

PENERAPAN

Contoh penerapan atas sebuah *array* sebagai data sumber yang akan diurutkan { 3, 9, 4, 1, 5, 2 } adalah sebagai berikut:

1. Pertama, dibagi menjadi dua bagian { 3, 9, 4 } dan { 1, 5, 2 }
2. Kemudian diurutkan secara terpisah sehingga menjadi { 3, 9, 4 } dan { 1, 5, 2 }
3. Sebuah wadah baru dibentuk sebagai penggabungan dari kedua *array* tersebut { 1 }, sementara nilai-nilai dalam masing *array* { 3, 4, 9 } dan { 2, 5 }

4. Langkah terakhir adalah penggabungan dari masing-masing *array* ke dalam wadah baru yang dibuat sebelumnya.

Contoh:

{ 1, 2 } <=> { 3, 4, 9 } dan { 5 }

{ 1, 2, 3 } <=> { 4, 9 } dan { 5 }

{ 1, 2, 3, 4 } <=> { 9 } dan { 5 }

{ 1, 2, 3, 4, 5 } <=> { 5 } dan { null }

{ 1, 2, 3, 4, 5, 9 } <=> { null } dan { null }

ALGORITMA DAN PSEUDOCODE

```
k = 0;

i = 0; j = 0;

While (i <= 4 && j <= 6) {

    If (A[i] <= B[j]) {

        C[k] = A[i];

        k++;

        i++;

    } Else {

        C[k] = B[j];

        k++;

        j++;

    }

}
```

HASIL

Jumlah data yang dimulai dari 10.000 buah dengan algoritma *Merge Sort*, karena jumlah data yang 10, 100 atau 1.000 tidak dapat dihitung *running-time* dalam detik atau milidetik (terlalu cepat dan jumlah data lebih sedikit).

Jumlah Data	MS (Milidetik)
10.000	0.156ms
20.000	0.234ms
30.000	0.344ms
40.000	0.437ms
50.000	0.593ms

IMPLEMENTASI

```
<body>

<div class="container">

  <h1>Mahasiswa List</h1>

  <div class="controls">

    <input type="number" id="student-nim" placeholder="NIM" />

    <input type="text" id="student-name" placeholder="Nama Lengkap" />

    <button onClick="addStudent()">Tambah</button>

  </div>

  <div class="sort-controls">

    <button onClick="sortStudents('asc')">Terkecil - Terbesar</button>

    <button onClick="sortStudents('desc')">Terbesar - Terkecil</button>

  </div>

  <ul id="student-list">

    <li data-nim="362379011">

      362379011 | Muhammad Cavin Hartono Putra

      <button class="delete-btn" onClick="deleteStudent(this)">

        Hapus

      </button>

    </li>

    <li data-nim="362379006">

      362379006 | Fikri Kharul Azhar

      <button class="delete-btn" onClick="deleteStudent(this)">

        Hapus

      </button>

    </li>

    <li data-nim="362379010">

      362379010 | Bintang Paditya

      <button class="delete-btn" onClick="deleteStudent(this)">

        Hapus

      </button>

    </li>

  </ul>
```

Source Code HTML untuk menampilkan halaman

```

function mergeSort(Arr, order = "asc") {
  if (Arr.length < 2) {
    return Arr;
  }

  const mid = Math.floor(Arr.length / 2);
  const left = Arr.slice(0, mid);
  const right = Arr.slice(mid);

  return merge(mergeSort(left, order), mergeSort(right, order), order);
}

function merge(left, right, order) {
  let result = [];
  let leftIndex = 0;
  let rightIndex = 0;

  while (leftIndex < left.length && rightIndex < right.length) {
    if (
      (order === "asc" && left[leftIndex].nim < right[rightIndex].nim) ||
      (order === "desc" && left[leftIndex].nim > right[rightIndex].nim)
    ) {
      result.push(left[leftIndex]);
      leftIndex++;
    } else {
      result.push(right[rightIndex]);
      rightIndex++;
    }
  }

  return result.concat(left.slice(leftIndex)).concat(right.slice(rightIndex));
}

```

Source Code JavaScript untuk mengurutkan data dengan Merge Sort


```

function addStudent() {

    const name = document.getElementById("student-name").value;
    const nim = document.getElementById("student-nim").value;

    if (name === "" || nim === "") {
        alert("Please enter both student name and NIM.");
        return;
    }

    const studentList = document.getElementById("student-list");
    const li = document.createElement("li");
    li.setAttribute("data-nim", nim);
    li.innerHTML = `${nim} | ${name} <button class="delete-btn" onclick="deleteStudent(this)">Delete</button>`;

    studentList.appendChild(li);

    // Clear input fields
    document.getElementById("student-name").value = "";
    document.getElementById("student-nim").value = "";
}

function deleteStudent(button) {
    const li = button.parentElement;
    li.remove();
}

```

Source Code JavaScript untuk menambahkan dan menghapus data